Declarative programming

LECT. DR. CALIN ALINA DELIA

Constraint Satisfaction Problems (CSP)

- Constraint Satisfaction Problem (CSP) example on zebra puzzle:
  - **Variables (V):** The items or positions you need to determine (e.g., *House 1, House 2, House 3...*, or *Person 1, Person 2...*).
  - **Domains (D):** The possible values each variable can take (e.g., *Colors: Red, Green, Blue, Yellow, Ivory*; *Pets: Dog, Snails, Fox, Horse, Zebra*).
  - **Constraints (C):** The rules that restrict which values can be assigned to which variables simultaneously. Common Types of Constraints
    - **Unary:** Relates to a single variable (e.g., X1 <> 3).
    - **Binary:** Relates two variables (e.g., X1>X2).
    - **Global / N-ary:** Relates multiple variables at once (e.g., alldifferent(X1, X2, X3).

Einstein Riddle Problem

- 0. There are three houses, three nationals, three animals, three cigarettes.
- 1. The Englishman lives in the first house on the left.
- 2. In the house immediately on the right of that housing the wolf, they smoke Luky Strike.
- 3. The Spaniard smokes Kent.
- 4. The Russian has a horse.
- Goals: Who smokes LM? Who has the dog?

Solution

- There are two solutions:

I. Englishman dog LM

Spaniard wolf Kent

Russian horse LS

II. Englishman wolf LM

Russian horse LS

Spaniard dog Kent

Prolog implementation - backtracking

```
solve(N, A, C) :-
  candidate(N, A, C),
  constraints(N, A, C),
  nl.

candidate(N, A, C) :-
  perm([eng,spa,rus], N),
  perm([dog,wol,hor], A),
  perm([Im,ken,Is], C).

constraints(N, A, C) :-
  pos(N, A, C, eng, _, _, p1),
  pos(N, A, C, _, wol, _, H1),
  posleft(H1, H2),
  pos(N, A, C, _, _, Is, H2),
  pos(N, A, C, spa, _, ken, _),
  pos(N, A, C, rus, hor, _, _).

pos([N1,_,_], [A1,_,_], [C1,_,_],
N1, A1, C1, p1).

pos([_,N2,_], [_,A2,_], [_,C2,_],
N2, A2, C2, p2).

pos([_,_,N3], [_,_,A3], [_,_,C3],
N3, A3, C3, p3).

perm([], []).
perm(Y, [A|X]) :-
   delete(A, Y, Y1),
   perm(Y1, X).

delete(A, [A|X], X).
delete(A, [B|X], [B|Y]) :-
   delete(A, X, Y).

posleft(p1, p2).
posleft(p2, p3).
```

The Zebra Puzzle (Einstain Problem)

- Implementation in Prolog of : Inspiring Women Puzzle
  - https://www.brainzilla.com/logic/zebra/inspiring-women/
  - Five girls are side by side talking about their inspirations. Each girl has woman in their family that they consider a role model. Follow the clues to find out which girl was inspired by her sister.
  - Rules (next slide)

- Solution Arc Consistency:
  - https://github.com/alinacalin/Scripts-Portfolio/tree/main/Prolog

- Jessie is exactly to the left of the girl that wants to become a Nurse.
- At the second position is the girl inspired by her Grandmother.
- Sabrina is 10 years old.
- Amy is next to the girl inspired by her Grandmother.
- The girl wearing the Green shirt is exactly to the left of the girl inspired by her Mother.
- The girl inspired by her Mother likes Grapefruit juice.
- The girl wearing the Red shirt is somewhere between the girl who likes Lemon juice and Amy, in that order.
- The girl that likes Pineapple juice is somewhere to the right of the girl wearing the Red shirt.
- The youngest girl is next to the girl who likes Orange juice.
- The girl inspired by her Cousin is exactly to the left of the girl wearing the Red shirt.
- The oldest girl is next to the girl wearing the Blue shirt.
- At one of the ends is the girl that likes Cranberry juice.
- The girl inspired by her Aunt is exactly to the right of the 11-year-old girl.
- Lindy is 11 years old.
- The 10-year-old girl is exactly to the right of the 8-year-old girl.
- The girl that wants to become a Nurse was inspired by her Grandmother.
- The girl wearing the Red shirt is somewhere between the girl that likes Lemon juice and the girl wearing the Yellow shirt, in that order.
- At the fifth position is the girl who wants to become a Writer.
- Lindy is exactly to the right of the 9-year-old girl.
- The girl that likes Pineapple juice is next to the girl that wants to become a Teacher.
- At the third position is the girl who wants to be a Journalist.
- The youngest girl is exactly to the right of Lindy.

For the Inspiring Women Puzzle

- CSP definition involves:
  - Variables:
  - Domains:
  - Constraints:

Common approaches for solving

Common Solving Techniques

Instead of using brute-force search across all potential combinations, algorithms solve CSPs by combining search with constraint propagation:

- **Backtracking Search:** A form of depth-first search that assigns values one variable at a time and backtracks as soon as a constraint is violated.
- **Arc Consistency (AC-3):** An algorithm that reduces the search space before or during search by eliminating domain values that cannot possibly satisfy binary constraints.
- **Forward Checking:** Tracks remaining valid values for unassigned variables and stops early if any variable runs out of valid domain options.

N Queens implementation in Prolog vs other languages

- CSP Definition:
  - Variables: The queens positions (eight in this case) {Q1, Q2, … Q8}
  - Domains: Positions in rows {1,2, … , 8} for each Qi, i=1, 8
  - Constraints:
    - Row : $Q_i \neq Q_j$ for any two distinct queens $Q_i$and Q_j(where $i \neq j$)
    - Diagonal: $|Q_i - Q_j| \neq |i - j|$
- Solutions:
  - https://www.geeksforgeeks.org/cpp/n-queen-problem-in-cpp/
  - https://www.geeksforgeeks.org/dsa/n-queen-problem-backtracking-3/

Prolog Backtracking version 1

```
queens_1(N,Qs) :-
    range(1,N,Rs),
    perm(Rs,Qs),
    test(Qs).

range(A,A,[A]).
range(A,B,[A|L]) :-
    A < B,
    A1 is A+1,
range(A1,B,L).
```

```
perm([],[]).
perm(Qs,[Y|Ys]) :-
    delete(Y,Qs,Rs),
    perm(Rs,Ys).

delete(X,[X|Xs],Xs).
delete(X,[Y|Ys],[Y|Zs]) :-
    delete(X,Ys,Zs).

member(E, [E|_]).
member(E, [_|L]) :-
member(E, L).
```

```
test(Qs) :-
    test(Qs,1,[],[]).
test([],_,_,_).
test([Y|Ys],X,Cs,Ds) :-
    C is X-Y,
    not(member(C,Cs)),
    D is X+Y,
    not(member(D,Ds)),
    X1 is X + 1, 5

test(Ys,X1,[C|Cs],[D|Ds]).
```

Prolog Version 2 AC

```
queens_2(N,Qs) :-
    range(1,N,Rs),
    perm_test(Rs,Qs,1,[],[]).

delete(X,[X|Xs],Xs).
delete(X,[Y|Ys],[Y|Zs]) :-
      delete(X,Ys,Zs).

member(E, [E|_]).
member(E, [_|L]) :-
member(E, L).
```

```
perm_test([],[],_,_,_).
perm_test(Qs,[Y|Ys],X,Cs,Ds) :-
     delete(Y,Qs,Rs),
     C is X-Y,
      not(member(C,Cs)),
      D is X+Y,
      not(member(D,Ds)),
      X1 is X+1,

perm_test(Rs,Ys,X1,[C|Cs],[D|Ds]).
```

Sudoku implementation in Prolog vs other languages

- CSP Definition:
  - Variables: 9X9 variables $X = \{V_{1,1}, V_{1,2}, \dots, V_{i,j}, \dots, V_{9,9}\}$
  - Domains:
    - **Empty cell:** $D_{i,j} = \{1,2,3,4,5,6,7,8,9\}$
    - **Pre-filled cell (given clue):** $D_{i,j} = \{k\}$where k is the fixed given digit.
  - Constraints:
    - For any two distinct variables $V_{a,b}$and $V_{c,d}$that share the same row, column, or 3 × 3 block:

$$V_{a,b} \neq V_{c,d}$$

- Solutions:
  - https://www.geeksforgeeks.org/dsa/sudoku-backtracking-7/
  - https://www.swi-prolog.org/pldoc/man?section=clpfd-sudoku

Prolog solution

```
sudoku(Rows) :-
    length(Rows, 9), maplist(same_length(Rows), Rows),
    append(Rows, Vs), Vs ins 1..9,
    maplist(all_distinct, Rows),
    transpose(Rows, Columns),
    maplist(all_distinct, Columns),
    Rows = [As,Bs,Cs,Ds,Es,Fs,Gs,Hs,Is],
    blocks(As, Bs, Cs),
    blocks(Ds, Es, Fs),
    blocks(Gs, Hs, Is).

blocks([], [], []).
blocks([N1,N2,N3|Ns1], [N4,N5,N6|Ns2], [N7,N8,N9|Ns3]) :-
    all_distinct([N1,N2,N3,N4,N5,N6,N7,N8,N9]),
    blocks(Ns1, Ns2, Ns3).
```

```
problem(1, [[_,_,_,_,_,_,_,_,_],
        [_,_,_,_,_,3,_,8,5],
        [_,_,1,_,2,_,_,_,_],
        [_,_,_,5,_,7,_,_,_],
        [_,_,4,_,_,_,1,_,_],
        [_,9,_,_,_,_,_,_,_],
        [5,_,_,_,_,_,_,7,3],
        [_,_,2,_,1,_,_,_,_],
        [_,_,_,_,4,_,_,_,9]]).
```

- ?- problem(1, Rows), sudoku(Rows), maplist(portray_clause, Rows).